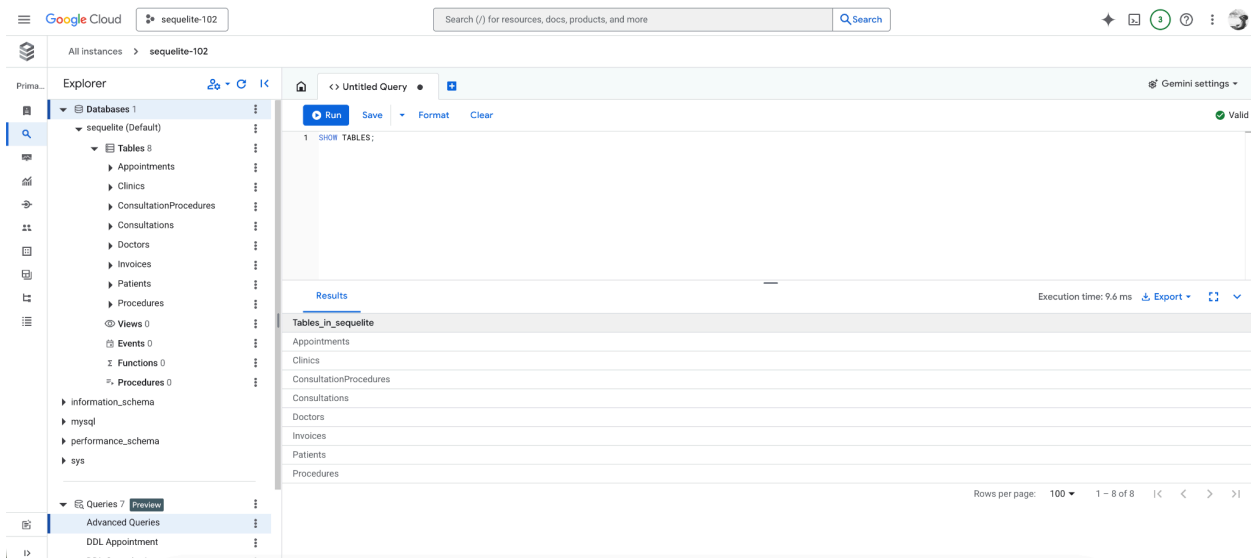


Digital Clinic Management System

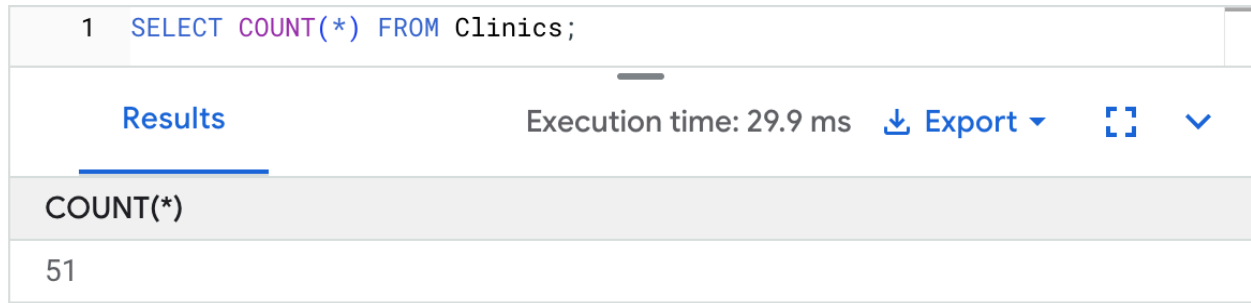
GCP Cloud SQL Implementation



DDL Commands

Clinics

```
CREATE TABLE Clinics(  
  ClinicID INT AUTO_INCREMENT PRIMARY KEY,  
  ClinicName VARCHAR(255) NOT NULL,  
  Zipcode CHAR(5) NOT NULL  
);
```



Doctors

```
CREATE TABLE Doctors(  
    DoctorID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(255) NOT NULL,  
    LastName VARCHAR(255),  
    EmailAddress VARCHAR(255) UNIQUE NOT NULL,  
    PasswordHash CHAR(255) NOT NULL,  
    Specialization VARCHAR(255),  
    ConsultationFees DECIMAL(10,2),  
    Role VARCHAR(10) NOT NULL,  
    ClinicID INT,  
    CHECK (Role IN ('admin', 'consultant')),  
    FOREIGN KEY (ClinicID) REFERENCES Clinics(ClinicID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM Doctors;	
Results	Execution time: 11.0 ms	Export SQL v
COUNT(*)		
1000		

Patients

```
CREATE TABLE Patients(  
    PatientID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(255) NOT NULL,  
    LastName VARCHAR(255),  
    EmailAddress VARCHAR(255),  
    Gender CHAR(1),  
    DateOfBirth DATE,  
    ClinicID INT,  
    CHECK (Gender IN ('M', 'F')),  
    FOREIGN KEY (ClinicID) REFERENCES Clinics(ClinicID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

```
1  SELECT COUNT(*) FROM Patients;
```

Results

Execution time: 31.4 ms

↓ Export ▾

⌂ ▾

COUNT(*)
1000

Appointments

```
CREATE TABLE Appointments(  
    AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
    AppointmentStatus CHAR(3) NOT NULL,  
    StartTime DATETIME NOT NULL,  
    EndTime DATETIME NOT NULL,  
    PatientID INT,  
    DoctorID INT,  
    CHECK (AppointmentStatus IN ('BKD','ACT','COM','CAN')),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE,  
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1 SELECT COUNT(*) FROM Appointments;	
Results	Execution time: 4.3 ms Export SQL v
COUNT(*)	
1784	

Consultations

```
CREATE TABLE Consultations(  
    ConsultationID INT AUTO_INCREMENT PRIMARY KEY,  
    AppointmentID INT,  
    HeartRate INT,  
    RespiratoryRate INT,  
    Temperature DECIMAL(4,1),  
    BloodOxygen DECIMAL(4,1),  
    SystolicBP INT,  
    DiastolicBP INT,  
    Weight DECIMAL(6,2),  
    Height DECIMAL(6,2),  
    ChiefComplaints TEXT,  
    Diagnosis TEXT,  
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM Consultations;
Results	Execution time: 8.1 ms Export SQL Close
COUNT(*)	
1784	

Procedures

```
CREATE TABLE Procedures(  
    ProcedureID INT AUTO_INCREMENT PRIMARY KEY,  
    ProcedureName VARCHAR(255) NOT NULL,  
    Amount DECIMAL(10,2) NOT NULL  
);
```

1	SELECT COUNT(*) FROM Procedures;
Results Execution time: 13.9 ms Export Full Screen Refresh	
COUNT(*)	
1001	

ConsultationProcedures

```
CREATE TABLE ConsultationProcedures(  
    ConsultationID INT,  
    ProcedureID INT,  
    PRIMARY KEY (ConsultationID, ProcedureID),  
    FOREIGN KEY (ConsultationID) REFERENCES Consultations(ConsultationID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,  
    FOREIGN KEY (ProcedureID) REFERENCES Procedures(ProcedureID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM ConsultationProcedures;
Results Execution time: 1.3 ms Export Full Screen Refresh	
COUNT(*)	
1000	

Invoices

```
CREATE TABLE Invoices(  
    InvoiceID INT AUTO_INCREMENT,  
    Amount DECIMAL(10, 2),  
    AppointmentID INT,  
    PRIMARY KEY (InvoiceID, AppointmentID),  
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

1	SELECT COUNT(*) FROM Invoices;	
Results	Execution time: 1.5 ms	↓ Export □ v
COUNT(*)		
1000		

Advanced SQL Queries

1. Get revenue for a clinic grouped by month, year

```
SELECT YEAR(EndTime) AS AccountYear, MONTH(EndTime) AS AccountMonth,
SUM(Amount)
FROM Appointments NATURAL JOIN Invoices NATURAL JOIN Doctors
WHERE AppointmentStatus = 'COM' AND ClinicID = 242
GROUP BY AccountYear, AccountMonth
ORDER BY AccountYear DESC, AccountMonth DESC;
```

Output (4 rows)

AccountYear	AccountMonth	SUM(Amount)
2022	6	3122623.00
2022	4	1606525.00
2021	1	1587342.00
2020	12	2281530.00

Indexing:

Before:

EXPLAIN
-> Sort: AccountYear DESC, AccountMonth DESC (actual time=1.01..1.01 rows=4 loops=1) -> Table scan on <temporary> (actual time=0.994..0.995 rows=4 loops=1) -> Aggregate using temporary table (actual time=0.993..0.993 rows=4 loops=1) -> Nested loop inner join (cost=111 rows=103) (actual time=0.0755..0.28 rows=16 loops=1) -> Nested loop inner join (cost=74.9 rows=103) (actual time=0.0615..0.196 rows=24 loops=1) -> Covering index lookup on Doctors using ClinicID (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0242..0.0367 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=1.46 rows=2.93) (actual time=0.00393..0.00436 rows=0.686 loops=35) -> Index lookup on Appointments using appointment_doctor_id (DoctorID=Doctors.DoctorID) (cost=1.46 rows=5.81) (actual time=0.0036..0.00395 rows=1.14 loops=35) -> Index lookup on Invoices using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.251 rows=1) (actual time=0.00296..0.00324 rows=0.667 loops=24)

After:

EXPLAIN
-> Sort: AccountYear DESC, AccountMonth DESC (actual time=0.535..0.535 rows=4 loops=1) -> Table scan on <temporary> (actual time=0.517..0.518 rows=4 loops=1) -> Aggregate using temporary table (actual time=0.515..0.515 rows=4 loops=1) -> Nested loop inner join (cost=82.1 rows=20.3) (actual time=0.0894..0.456 rows=16 loops=1) -> Nested loop inner join (cost=74.9 rows=20.3) (actual time=0.0743..0.31 rows=24 loops=1) -> Covering index lookup on Doctors using ClinicID (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0274..0.0538 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=1.45 rows=0.581) (actual time=0.00604..0.00706 rows=0.686 loops=35) -> Index lookup on Appointments using appointment_doctor_id (DoctorID=Doctors.DoctorID) (cost=1.45 rows=5.81) (actual time=0.00565..0.00615 rows=1.14 loops=35) -> Index lookup on Invoices using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.255 rows=1) (actual time=0.00482..0.00556 rows=0.667 loops=24)

Added an index for the column **AppointmentStatus** on **Appointments** table.

- Cost before indexing to filter = 1.46.

- Cost after indexing to filter = 1.45.
- No significant change in costs after indexing, but still added it as it would be useful for other simple queries in our application. It might be possible due to a majority of the appointments in our data being completed (i.e. AppointmentStatus = COM) leading to not much of a change with or without indexing.

Added an index for the column **EndTime** on **Appointments** table, as it was used for grouping and sorting. (no cost metrics for these operations mentioned in the EXPLAIN ANALYZE, derived after running multiple times)

- Average aggregation time before indexing = 1 seconds.
- Average aggregation time after indexing = 0.5 seconds.
- Sorting time reduced from 2.1 seconds to 0.5 seconds on average.

Added an index for the column **DoctorID** on **Appointments** table, which was used for intermediate joining to get to the clinic from the appointment.

- SQL already picks the index from the **Doctors** table since it was a foreign key, so no change in costs.

2. Get patient visits for a clinic grouped by month, year

```
SELECT YEAR(EndTime) AS VisitYear, MONTH(EndTime) AS VisitMonth,
COUNT(AppointmentID)
FROM Appointments NATURAL JOIN Doctors
WHERE AppointmentStatus <> 'CAN' AND ClinicID = 242
GROUP BY VisitYear, VisitMonth
ORDER BY VisitYear DESC, VisitMonth DESC;
```

Output (9 rows)

VisitYear	VisitMonth	COUNT(AppointmentID)
2023	12	4
2023	9	4
2022	6	4
2022	4	4
2022	1	4
2021	5	4
2021	3	4
2021	1	8
2020	12	4

Indexing:

Before:

EXPLAIN
-> Sort: VisitYear DESC, VisitMonth DESC (actual time=0.349..0.35 rows=9 loops=1) -> Table scan on <temporary> (actual time=0.329..0.33 rows=9 loops=1) -> Aggregate using temporary table (actual time=0.327..0.327 rows=9 loops=1) -> Nested loop inner join (cost=74.9 rows=203) (actual time=0.0607..0.279 rows=40 loops=1) -> Covering index lookup on Doctors using ClinicID (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0231..0.0434 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus <> 'CAN') (cost=1.47 rows=5.81) (actual time=0.00567..0.00642 rows=1.14 loops=35) -> Index lookup on Appointments using appointment_doctor_id (DoctorID=Doctors.DoctorID) (cost=1.47 rows=5.81) (actual time=0.00543..0.00596 rows=1.14 loops=35)

After:

EXPLAIN
-> Sort: VisitYear DESC, VisitMonth DESC (actual time=0.304..0.305 rows=9 loops=1) -> Table scan on <temporary> (actual time=0.292..0.293 rows=9 loops=1) -> Aggregate using temporary table (actual time=0.29..0.29 rows=9 loops=1) -> Nested loop inner join (cost=74.9 rows=183) (actual time=0.056..0.25 rows=40 loops=1) -> Covering index lookup on Doctors using ClinicID (ClinicID=242) (cost=3.76 rows=35) (actual time=0.0267..0.045 rows=35 loops=1) -> Filter: (Appointments.AppointmentStatus <> 'CAN') (cost=1.47 rows=5.23) (actual time=0.00482..0.00554 rows=1.14 loops=35) -> Index lookup on Appointments using appointment_doctor_id (DoctorID=Doctors.DoctorID) (cost=1.47 rows=5.81) (actual time=0.00457..0.00508 rows=1.14 loops=35)

Utilized previously added index from Advanced Query 1 for the column **AppointmentStatus** on **Appointments** table.

- Cost before indexing to filter = 1.47.
- Cost after indexing to filter = 1.47.
- No change in costs after indexing, but still added it as it would be useful for other simple queries in our application.

Utilized previously added index from Advanced Query 1 for the column **EndTime** on **Appointments** table, as it was used for grouping and sorting. (no cost metrics for these operations mentioned in the EXPLAIN ANALYZE, derived after running multiple times)

- Average aggregation time before indexing = 0.34 seconds.
- Average aggregation time after indexing = 0.29 seconds.
- Sorting time reduced from 0.37 seconds to 0.33 seconds on average.

Utilized previously added index from Advanced Query 1 for the column **DoctorID** on **Appointments** table, which was used for intermediate joining to get to the clinic from the appointment.

- SQL already picks the index from the **Doctors** table since it was a foreign key, so no change in costs.

3. Get total invoice amount for a particular appointment

```
SELECT SUM(OrderItemAmounts) AS InvoiceAmount
FROM (
  (SELECT Amount AS OrderItemAmounts
   FROM ConsultationProcedures NATURAL JOIN Procedures
   WHERE ConsultationID = 425)
 UNION ALL
  (SELECT ConsultationFees AS OrderItemAmounts
   FROM Doctors
   WHERE DoctorID = 170)
) AS OrderItems;
```

Output (1 row)

InvoiceAmount
1142.57

Indexing:

Before:

EXPLAIN
-> Aggregate: sum(OrderItems.OrderItemAmounts) (cost=3.62..3.62 rows=1) (actual time=0.177..0.177 rows=1 loops=1) -> Table scan on OrderItems (cost=2.16..3.42 rows=2) (actual time=0.169..0.17 rows=2 loops=1) -> Union all materialize (cost=0.9..0.9 rows=2) (actual time=0.167..0.167 rows=2 loops=1) -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.132..0.137 rows=1 loops=1) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=425) (cost=0.35 rows=1) (actual time=0.0373..0.0422 rows=1 loops=1) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.35 rows=1) (actual time=0.0933..0.0934 rows=1 loops=1) -> Rows fetched before execution (cost=0..0 rows=1) (actual time=116e-6..168e-6 rows=1 loops=1)

After:

EXPLAIN
-> Aggregate: sum(OrderItems.OrderItemAmounts) (cost=3.62..3.62 rows=1) (actual time=0.177..0.177 rows=1 loops=1) -> Table scan on OrderItems (cost=2.16..3.42 rows=2) (actual time=0.169..0.17 rows=2 loops=1) -> Union all materialize (cost=0.9..0.9 rows=2) (actual time=0.167..0.167 rows=2 loops=1) -> Nested loop inner join (cost=0.7 rows=1) (actual time=0.132..0.137 rows=1 loops=1) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=425) (cost=0.35 rows=1) (actual time=0.0373..0.0422 rows=1 loops=1) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.35 rows=1) (actual time=0.0933..0.0934 rows=1 loops=1) -> Rows fetched before execution (cost=0..0 rows=1) (actual time=116e-6..168e-6 rows=1 loops=1)

Added an index for the column **ConsultationFees** on **Doctors** table.

- No change in costs after indexing, since DoctorID was already indexed.

4. Get most performed procedures for a doctor

```
SELECT ProcedureName, COUNT(ProcedureID)
FROM ConsultationProcedures NATURAL JOIN Procedures NATURAL JOIN Consultations
NATURAL JOIN Appointments
WHERE AppointmentStatus = 'COM' AND DoctorID = 170
GROUP BY ProcedureName
HAVING COUNT(ProcedureID) > 0
ORDER BY TimesPerformed DESC;
```

Output (8 rows)

ProcedureName	TimesPerformed
Polypharmacy management	1
Interventional radiology procedures	1
Sedation for minor procedures	1
Plastic surgery	1
Cochlear implant surgery	1
Emergency surgical procedures	1
Comprehensive geriatric assessment	1
Allergy testing	1

Indexing:

Before:

EXPLAIN
-> Sort: TimesPerformed DESC (actual time=1.58..1.58 rows=8 loops=1) -> Filter: ('count(ConsultationProcedures.ProcedureID)' > 0) (actual time=1.57..1.57 rows=8 loops=1) -> Table scan on <temporary> (actual time=1.56..1.57 rows=8 loops=1) -> Aggregate using temporary table (actual time=1.56..1.56 rows=8 loops=1) -> Nested loop inner join (cost=11.6 rows=8) (actual time=1.02..1.48 rows=8 loops=1) -> Nested loop inner join (cost=8.76 rows=8) (actual time=1.01..1.46 rows=8 loops=1) -> Nested loop inner join (cost=5.96 rows=8) (actual time=1..1.41 rows=12 loops=1) -> Filter: ((Appointments.DoctorID = 170) and (Appointments.AppointmentStatus = 'COM')) (cost=3.16 rows=8) (actual time=0.963..1.35 rows=12 loops=1) -> Intersect rows sorted by row ID (cost=3.16 rows=8.07) (actual time=0.958..1.34 rows=12 loops=1) -> Index range scan on Appointments using appointment_doctor_id over (DoctorID = 170) (cost=0.635 rows=16) (actual time=0.898..0.903 rows=16 loops=1) -> Index range scan on Appointments using appointment_status over (AppointmentStatus = 'COM') (cost=1.72 rows=900) (actual time=0.036..0.39 rows=849 loops=1) -> Covering index lookup on Consultations using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.263 rows=1) (actual time=0.0034..0.00381 rows=1 loops=12) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=Consultations.ConsultationID) (cost=0.263 rows=1) (actual time=0.00302..0.00337 rows=0.667 loops=12) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.263 rows=1) (actual time=0.00248..0.00251 rows=1 loops=8)

After:

EXPLAIN
<pre>-> Sort: TimesPerformed DESC (actual time=0.27..0.271 rows=8 loops=1) -> Filter: ('count(ConsultationProcedures.ProcedureID)' > 0) (actual time=0.253..0.256 rows=8 loops=1) -> Table scan on <temporary> (actual time=0.251..0.252 rows=8 loops=1) -> Aggregate using temporary table (actual time=0.249..0.249 rows=8 loops=1) -> Nested loop inner join (cost=5.84 rows=1.6) (actual time=0.0985..0.217 rows=8 loops=1) -> Nested loop inner join (cost=5.28 rows=1.6) (actual time=0.091..0.193 rows=8 loops=1) -> Nested loop inner join (cost=4.72 rows=1.6) (actual time=0.0818..0.139 rows=12 loops=1) -> Filter: (Appointments.AppointmentStatus = 'COM') (cost=4.16 rows=1.6) (actual time=0.0691..0.0763 rows=12 loops=1) -> Index lookup on Appointments using appointment_doctor_id (DoctorID=170) (cost=4.16 rows=16) (actual time=0.0626..0.0668 rows=16 loops=1) -> Covering index lookup on Consultations using AppointmentID (AppointmentID=Appointments.AppointmentID) (cost=0.312 rows=1) (actual time=0.00337..0.00402 rows=1 loops=12) -> Covering index lookup on ConsultationProcedures using PRIMARY (ConsultationID=Consultations.ConsultationID) (cost=0.312 rows=1) (actual time=0.00371..0.00423 rows=0.667 loops=12) -> Single-row index lookup on Procedures using PRIMARY (ProcedureID=ConsultationProcedures.ProcedureID) (cost=0.312 rows=1) (actual time=0.00261..0.00265 rows=1 loops=8)</pre>

Added index for the column **ProcedureName** on **Procedures** table, as it was used for grouping.
(no cost metrics for these operations mentioned in the EXPLAIN ANALYZE, derived after running multiple times)

- Average aggregation time before indexing = 1.5 seconds.
- Average aggregation time after indexing = 0.3 seconds.

Utilized previously added index from Advanced Query 1 for the column **DoctorID** on **Appointments** table, which was used for intermediate joining to get to the clinic from the appointment.

- SQL already picks the index from the **Doctors** table since it was a foreign key, so no change in costs.

Final Indexing

The final indexes (apart from primary keys) added after analyzing the advanced queries and other use cases are as follows:

- **AppointmentStatus** on **Appointments** table (*Advanced queries 1, 2, 4 and fetching appointments by status on dashboard*)
- **EndTime** on **Appointments** table (*Advanced queries 1, 2*)
- **(StartTime, EndTime)** on **Appointments** table (*For fetching appointments by date on dashboard and patient profile*)
- **(FirstName, LastName)** on **Patients** table (*For faster patient search*)
- **ProcedureName** on **Procedures** table (*Advanced query 3 and for faster procedure search by name during consultation*)